Unit-IV: Network Management and Software Defined Networking (SDN) for IoT

Syllabus: Introduction to SDN; SDN Architecture (Controller, Data Plane, Control Plane); Benefits of SDN in IoT; SDN Models (Open SDN, API SDN, Overlay, Hybrid); Data Handling, Analytics and Security Considerations in SDN for IoT; Network Management Challenges in IoT; Integration of SDN with IoT Networks.

1. Introduction to Software Defined Networking (SDN)

1.1 Traditional Networking – the problem

In a traditional network, every device (router, switch) has two jobs bundled inside the same box: it decides where traffic should go (control logic) and it actually forwards the packets (forwarding logic). Each device is configured individually and runs its own routing protocols (OSPF, BGP, etc.). The network is therefore:

- Hard to configure and manage when there are hundreds or thousands of devices.
- Vendor-locked, because the control software is closed and proprietary.
- Slow to change; a new policy needs manual changes on many devices.
- Unsuitable for IoT, where millions of heterogeneous devices join and leave dynamically.

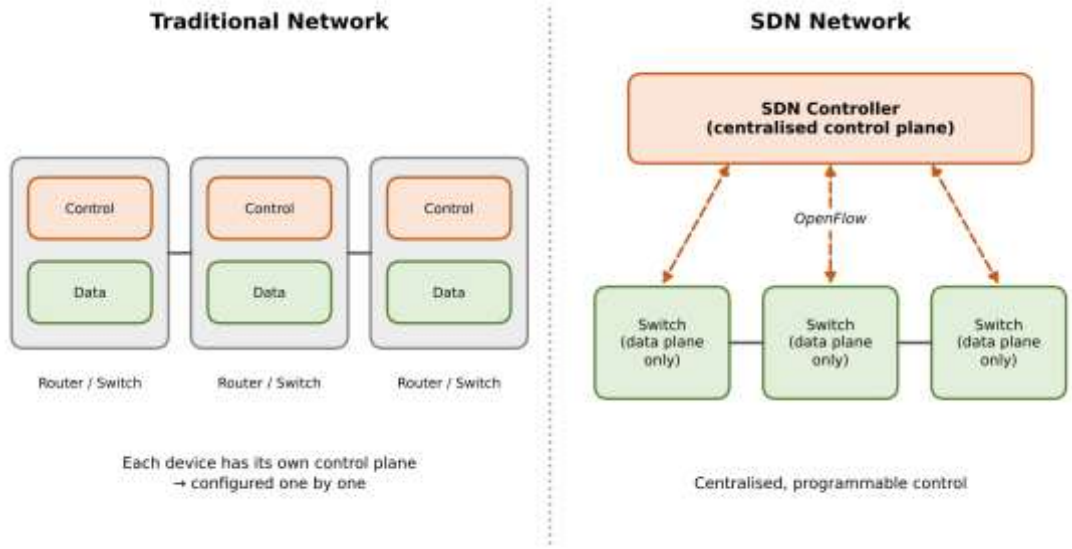


Fig 1.1: Traditional Network vs SDN Network (Distributed vs Centralized Control)

1.2 What is SDN?

Software Defined Networking (SDN) is a network architecture in which the control plane is separated (decoupled) from the data plane, and the network is controlled centrally through software. The network intelligence is logically centralised in an SDN controller that has a global view of the whole network. Switches become simple packet-forwarding devices that follow rules given by the controller.

Key principles of SDN

- **Separation of planes:** control logic is moved out of the forwarding devices.
- **Centralised control:** one logical controller sees and manages the entire network.
- **Programmability:** the network behaviour is defined by software/applications through open APIs.
- **Open standards:** protocols such as OpenFlow allow any vendor's hardware to be controlled.

Standardisation

SDN is promoted by the Open Networking Foundation (ONF). OpenFlow is the most popular protocol for communication between the controller and the switches.

1.3 Why SDN matters for IoT

IoT networks contain a huge number of constrained, heterogeneous devices using different protocols (Zigbee, BLE, LoRa, Wi-Fi, 6LoWPAN) and generating very different kinds of traffic. SDN gives a single programmable point from which this diversity can be managed, monitored and secured.

2. SDN Architecture

The SDN architecture is divided into three layers (planes) connected by two types of interfaces. The planes are: Application Plane (top), Control Plane (middle) and Data Plane (bottom).

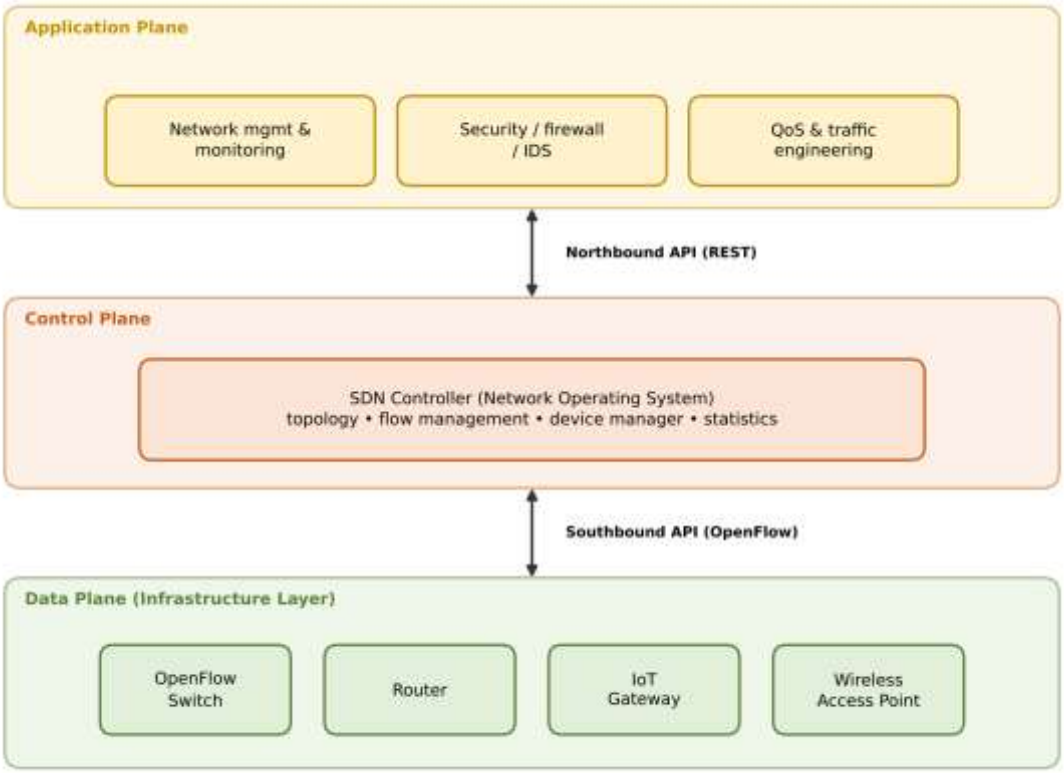


Fig 2.1: Three-Layer SDN Architecture

2.1 Data Plane (Infrastructure Layer)

The data plane consists of the physical and virtual forwarding devices: OpenFlow-enabled switches, routers, wireless access points and IoT gateways. Its job is only to forward, drop or modify packets as per the instructions received from the controller.

- Contains flow tables, where each entry has match fields, actions and counters.
- If an incoming packet matches a flow entry, the action is applied (forward to port, drop, modify header).
- If no match is found (table-miss), the packet is sent to the controller to decide what to do (Packet-In message).
- Collects statistics (packet and byte counts) and reports them to the controller.

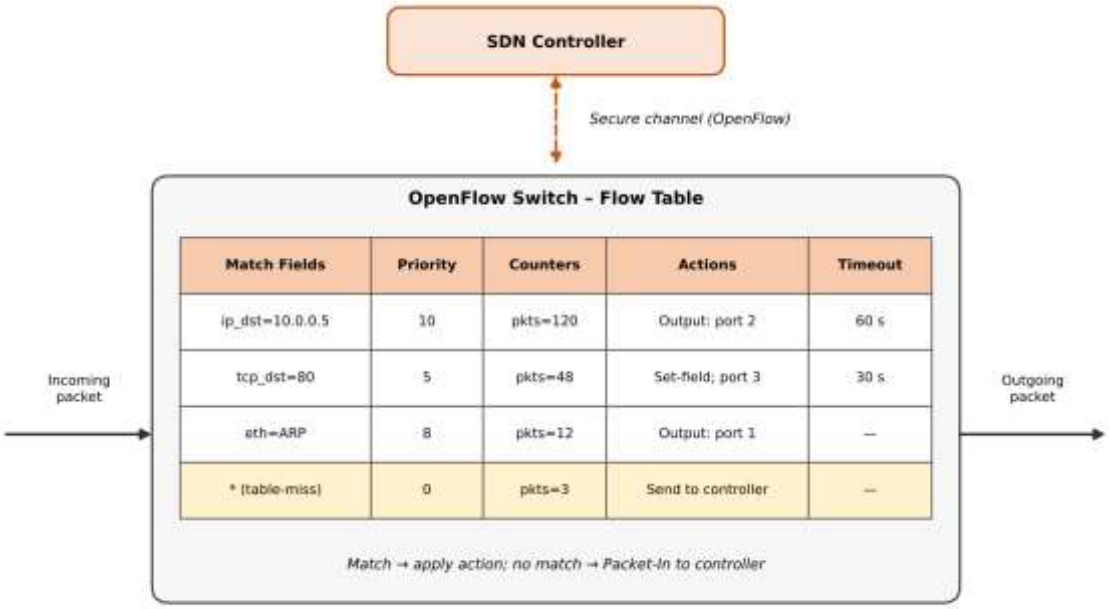


Fig 2.2: OpenFlow Switch and Flow Table Structure

2.2 Control Plane

The control plane is the 'brain' of the network. It decides how traffic should flow and installs those decisions in the data plane as flow rules. It is implemented by the SDN controller (or a cluster of controllers).

- Maintains the global network view (topology, link status, device list).
- Computes paths and policies (routing, load balancing, QoS).
- Programs the flow tables of switches through the southbound interface.
- Provides services to applications through the northbound interface.

2.3 SDN Controller

The SDN controller is the central software entity that sits between the applications and the network devices. It is often called the 'Network Operating System' (NOS).

Main functions

- Topology discovery and device management.
- Flow rule installation, modification and deletion.
- Network state and statistics collection.
- Providing abstraction and APIs to applications.
- Handling failures and rerouting traffic.

Popular controllers

Controller	Remarks
OpenDaylight (ODL)	Open-source, Java-based, modular; widely used in industry.
ONOS	Designed for carrier-grade, distributed and scalable networks.
Ryu	Python-based, simple, popular in student labs and research.
Floodlight	Java-based, easy to use, from Big Switch Networks.
POX / NOX	Early Python/C++ controllers for learning and prototyping.

Interfaces of the controller

- **Northbound API (NBI):** between controller and applications; usually REST APIs. Used by apps for routing, security, monitoring.
- **Southbound API (SBI):** between controller and switches; OpenFlow, NETCONF, OVSDB, SNMP etc.
- **East–West API:** between multiple controllers for synchronisation in distributed deployments.

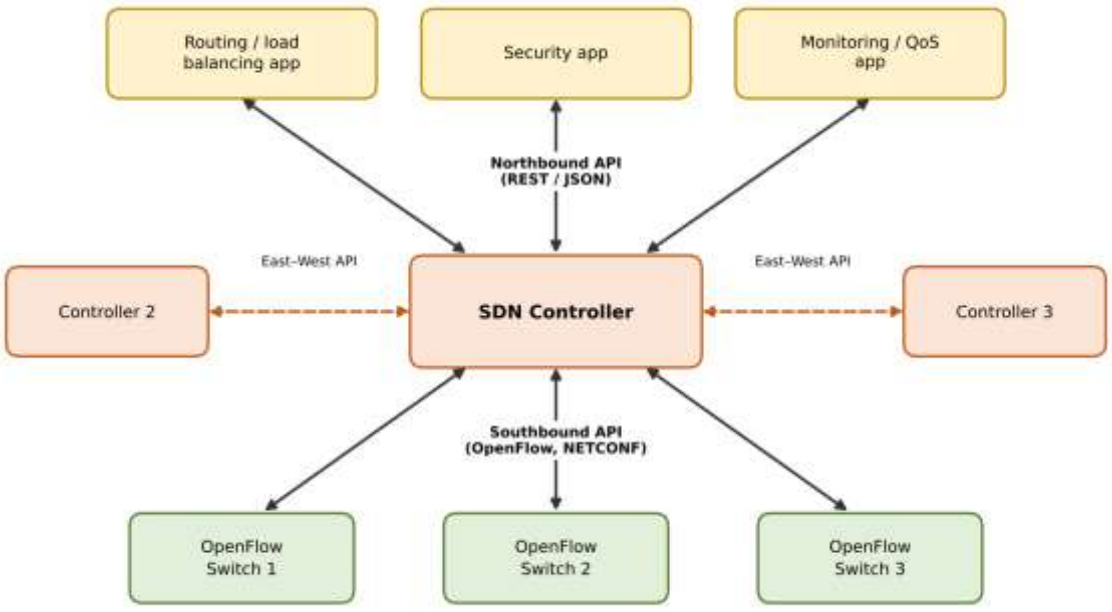


Fig 2.3: SDN Controller Interfaces (Northbound, Southbound, East-West)

Controller placement

A single controller is a bottleneck and a single point of failure. So large networks use multiple controllers in a distributed or hierarchical (cluster) arrangement for scalability and reliability.

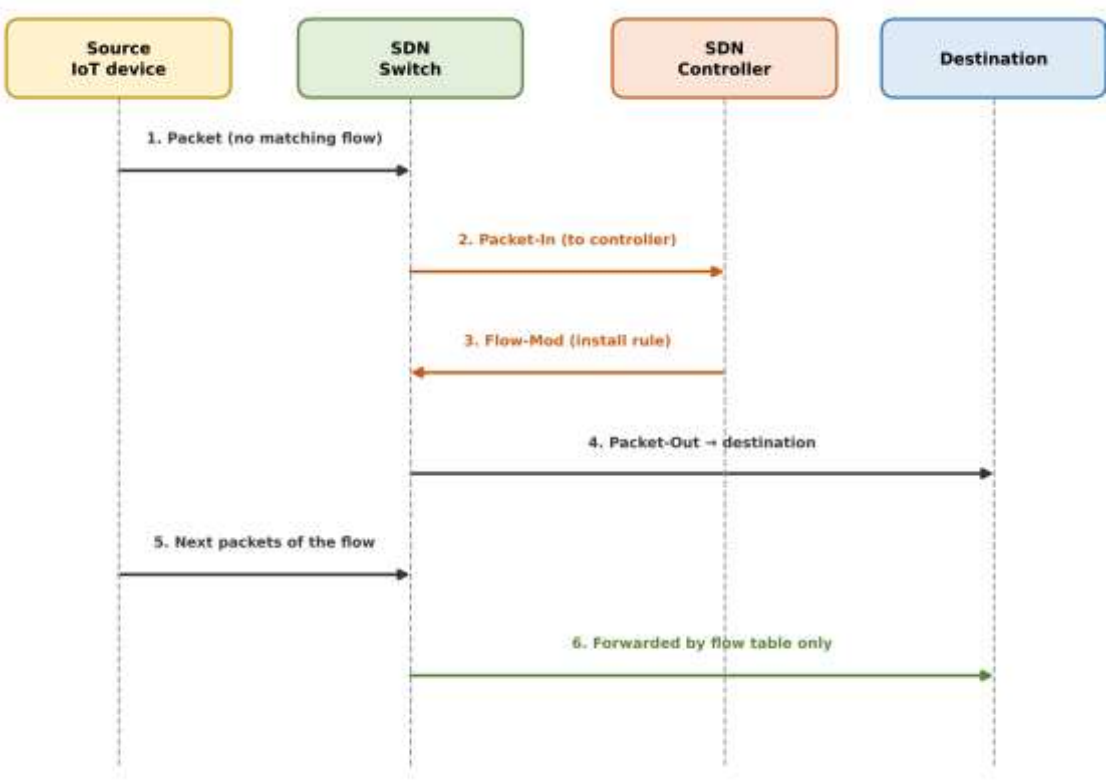


Fig 2.4: OpenFlow Message Flow: Packet-In, Flow-Mod and Packet-Out

2.4 Control Plane vs Data Plane – quick comparison

Aspect	Control Plane	Data Plane
Function	Decides the path	Forwards the packets
Intelligence	Centralised in controller	Simple, rule-following
Speed	Slower (software)	Fast (hardware, line-rate)
Examples	Routing, topology, policy	Switching, filtering, header rewrite
Communication	Southbound API (OpenFlow)	Receives rules via flow tables

3. Benefits of SDN in IoT

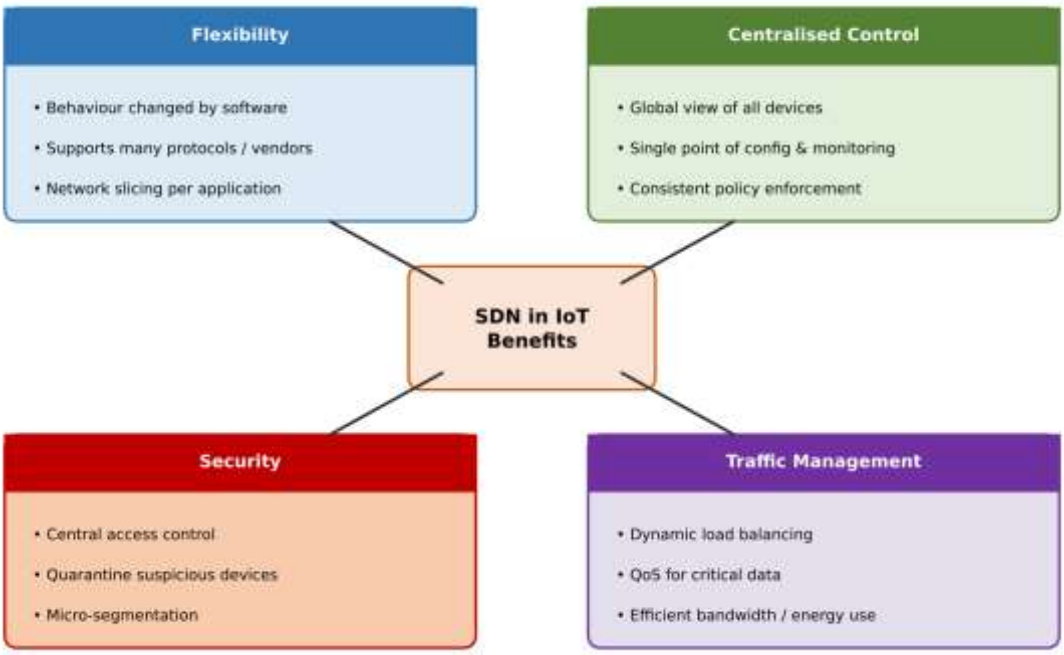


Fig 3.1: Benefits of SDN in IoT (Overview Diagram)

3.1 Flexibility

- Network behaviour can be changed by software without touching hardware.
- New IoT devices and services can be added quickly using programmable policies.
- Supports heterogeneous protocols and vendors, since the controller abstracts the hardware.
- Network slicing/virtualisation: separate virtual networks for different applications (e.g., healthcare, smart home) on the same infrastructure.

3.2 Centralised Control

- Global view of all devices, links and traffic from a single point.
- One place to configure, monitor and troubleshoot – reduces human error and operating cost.
- Consistent policy enforcement across the whole IoT network.
- Faster fault detection and recovery.

3.3 Security

- Controller can apply security policies uniformly (access control, isolation of devices).
- Suspicious devices or flows can be detected and blocked or quarantined instantly by changing flow rules.
- Enables micro-segmentation: compromised sensors cannot reach critical systems.

- Easy integration of firewalls, IDS/IPS as software applications (Network Function Virtualisation).

3.4 Traffic Management

- Dynamic load balancing and rerouting based on real-time network state.
- QoS: priority to critical traffic (e.g., patient monitoring, alarms) over bulk sensor data.
- Efficient use of limited bandwidth and energy, which is important for low-power IoT networks.
- Congestion control and traffic engineering using flow-level statistics.

3.5 Other advantages

- Reduced cost (commodity hardware, less manual work).
- Innovation and fast prototyping of new protocols.
- Better scalability for large IoT deployments.

4. SDN Models

SDN can be deployed in different ways depending on how much control is moved to the central controller and how the existing network is used. The four common models are:

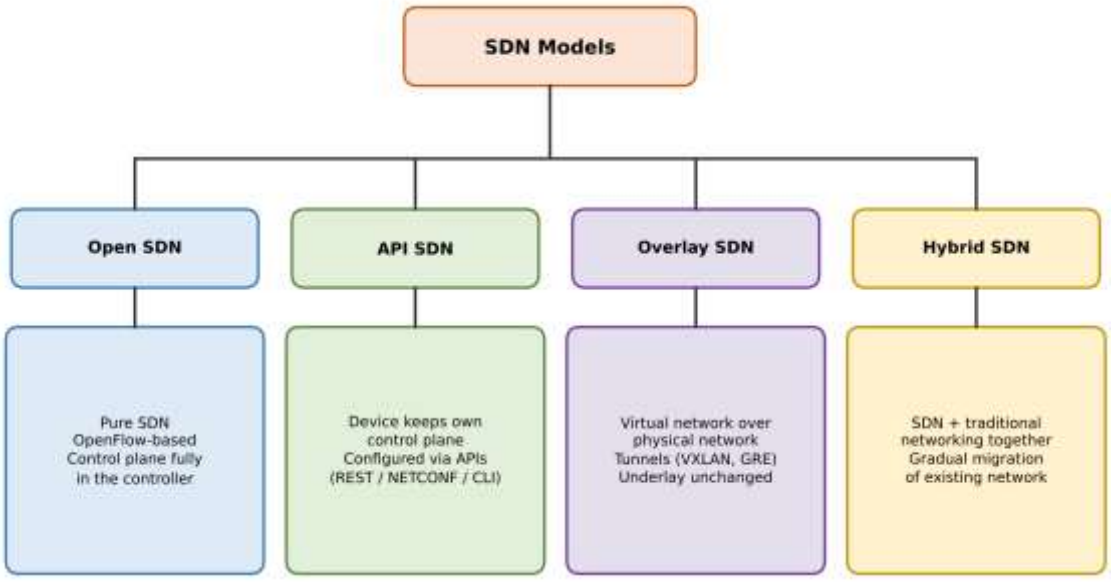


Fig 4.1: Classification of SDN Models

4.1 Open SDN

- Uses open protocols such as OpenFlow to control the forwarding behaviour of switches directly.
- Control plane is fully removed from devices and placed in the controller (pure SDN).
- Advantages: vendor independence, true programmability, global view.

- Disadvantages: needs OpenFlow-enabled hardware, controller is a critical point, hard to migrate existing networks.

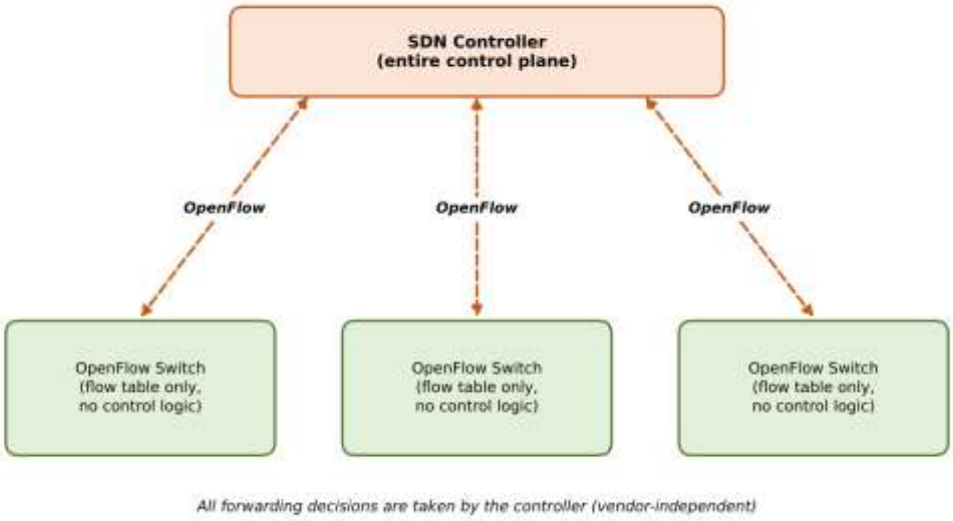


Fig 4.2: Open SDN Model

4.2 API SDN (SDN via APIs)

- Devices keep their own control plane, but vendors expose programmable APIs (REST, NETCONF, CLI scripting, SNMP) to configure them.
- Controller/application uses these APIs to push configuration to the devices.
- Advantages: works on existing hardware, easy to adopt, limited risk.
- Disadvantages: vendor-specific, less fine-grained control than OpenFlow, no complete decoupling.

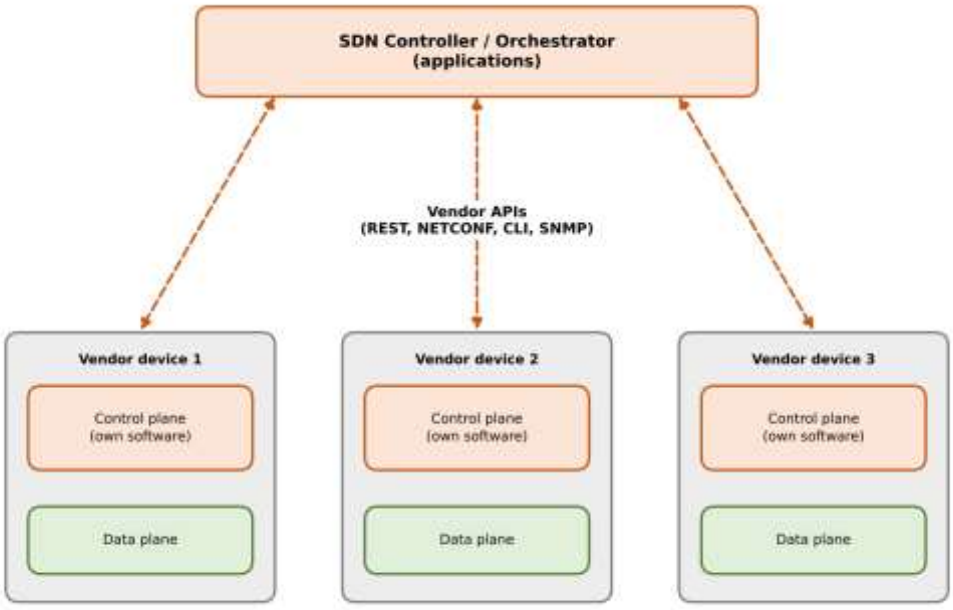


Fig 4.3: API-Based SDN Model

4.3 Overlay SDN

- A virtual network is built on top of the existing physical network using tunnelling (VXLAN, GRE, STT).
- Physical underlay is left unchanged; SDN controls only the virtual switches (e.g., Open vSwitch) at the edge/hosts.
- Advantages: no change to the physical network, good for multi-tenant data centres and cloud-based IoT platforms.
- Disadvantages: limited visibility of the underlay, extra encapsulation overhead.

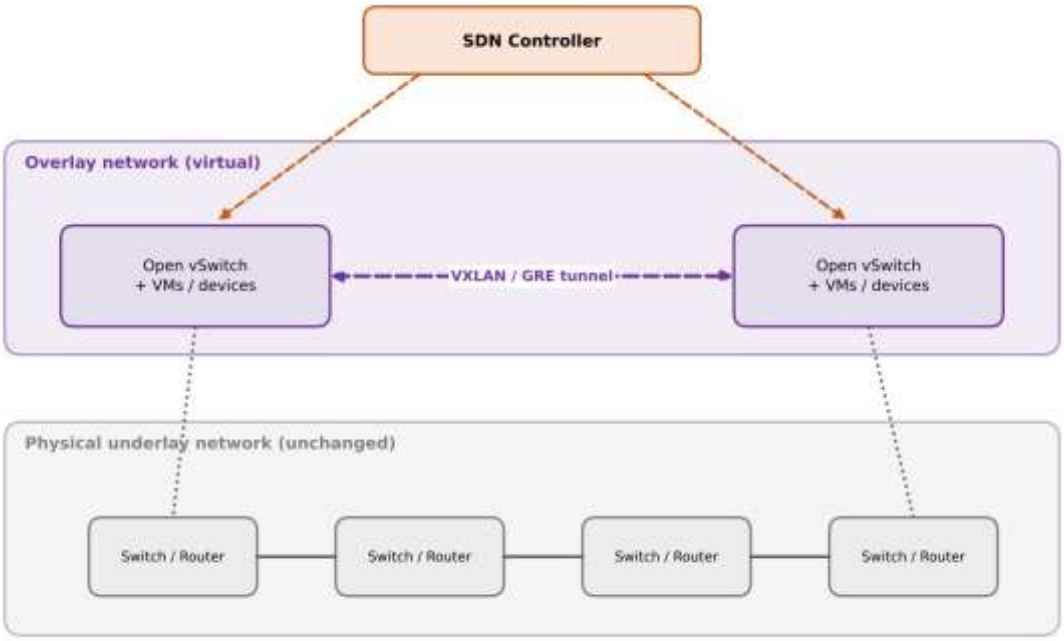


Fig 4.4: Overlay SDN Model

4.4 Hybrid SDN

- Combines traditional networking and SDN in the same network; both coexist.
- Some switches/regions follow SDN control, others continue with distributed protocols (OSPF/BGP).
- Advantages: gradual migration, protects existing investment, uses the best of both.
- Disadvantages: complex to manage, needs coordination between the two control mechanisms.

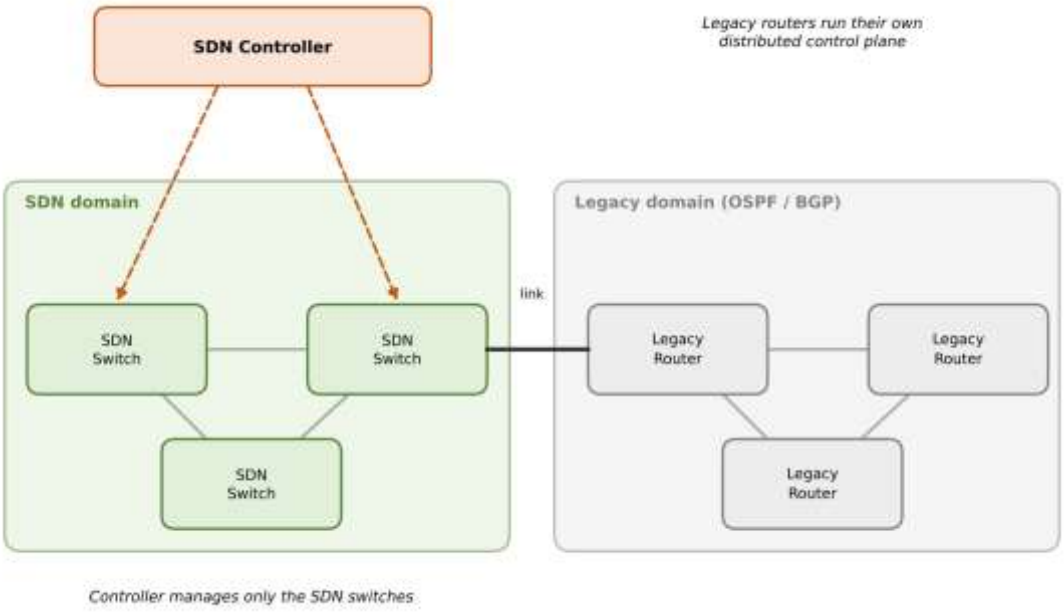


Fig 4.5: Hybrid SDN Model

4.5 Comparison of SDN models

Model	Control location	Requirement	Best suited for
Open SDN	Fully in controller	OpenFlow hardware	New / greenfield networks, research
API SDN	Still in devices; configured via APIs	Programmable device APIs	Existing enterprise networks
Overlay SDN	Virtual switches / edge	Tunnelling support	Data centres, cloud IoT
Hybrid SDN	Shared (controller + devices)	Both types of devices	Gradual migration of networks

5. Data Handling and Analytics in SDN for IoT

5.1 Data handling

IoT generates a very large volume of data with high velocity and variety. In an SDN-based IoT network, the controller handles data in the following ways:

- Flow-based data collection: switches keep per-flow counters (packets, bytes, duration) which the controller polls or receives through event messages.
- Data aggregation and filtering: rules are installed at gateways/switches to filter redundant sensor data and forward only useful data, saving bandwidth and energy.
- Prioritised forwarding: time-critical data (alarms, medical data) is given higher priority than periodic readings.
- Edge/fog processing: SDN directs selected flows to edge or fog nodes for local processing, and others to the cloud.

- Data steering: flows can be redirected to middleboxes (firewall, IDS, cache) as required (service chaining).

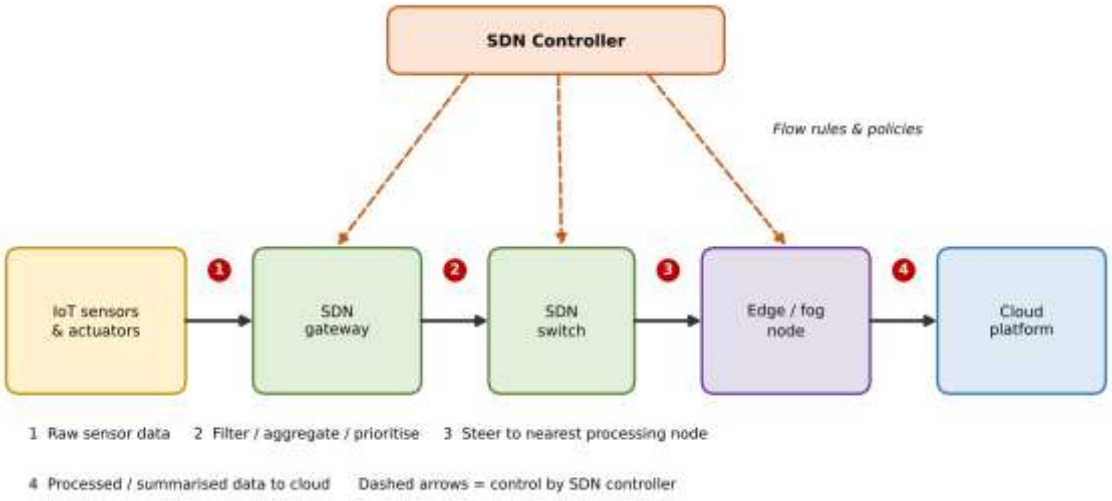


Fig 5.1: Data Flow in an SDN-based IoT Network

5.2 Analytics

Since the controller has a global view and receives network statistics continuously, it is an ideal place to run analytics.

- Network analytics: traffic pattern analysis, bandwidth usage, latency, packet loss, device behaviour.
- Predictive analytics: forecasting congestion or failures with machine learning and reconfiguring the network before problems occur.
- Anomaly detection: identifying abnormal device behaviour (for example a sensor sending too much data – a possible attack).
- Real-time monitoring and visualisation through dashboards; supports closed-loop automation (monitor → analyse → act).
- Big-data tools (Hadoop, Spark) and ML models may be integrated through the northbound API.

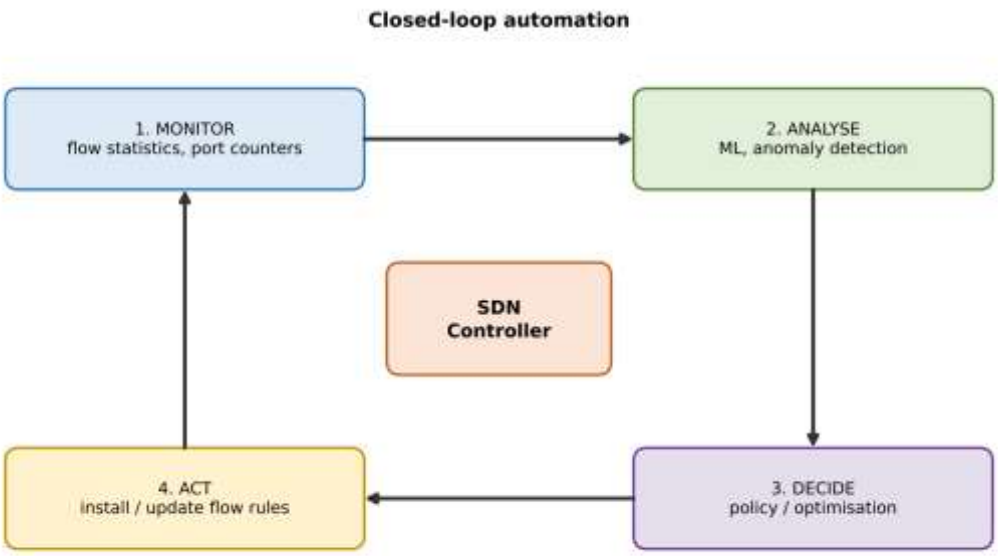


Fig 5.2: SDN Analytics Closed-Loop (Monitor – Analyse – Decide – Act)

6. Security Considerations in SDN for IoT

SDN improves security through central visibility and control, but the architecture itself introduces new attack surfaces that must be protected.

6.1 Threats to SDN-based IoT networks

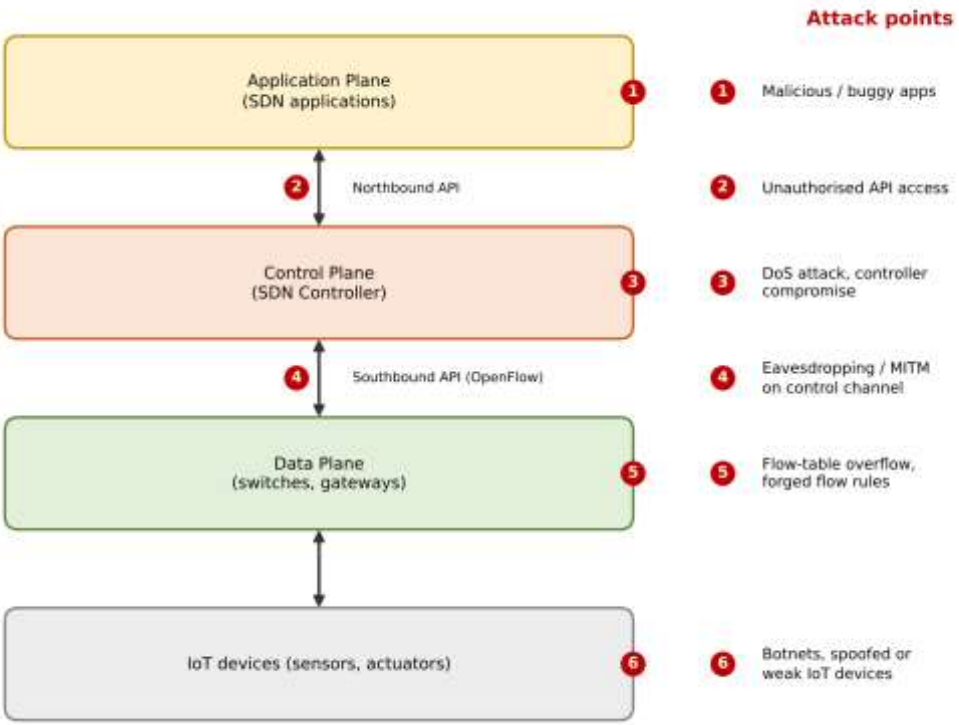


Fig 6.1: Attack Points in the SDN Architecture

Target	Possible attack / vulnerability
Controller	DoS/DDoS attack, compromise of the controller = control of the whole network (single point of failure).
Southbound channel	Eavesdropping, man-in-the-middle attack, spoofing of OpenFlow messages if TLS is not used.
Northbound API	Malicious or buggy applications, unauthorised API access.
Data plane (switches)	Flow-table overflow, packet-in flooding, forged flow rules.
IoT devices	Weak authentication, botnets (e.g. Mirai) which flood the network, spoofed device identities.

6.2 Security measures

- Secure channel: TLS/SSL between controller and switches.
- Authentication and authorisation of devices, applications and administrators (role-based access control).
- Controller redundancy and clustering for availability and failover.
- Rate limiting and flow-table protection to counter DoS and Packet-In flooding.
- Intrusion detection/prevention and anomaly detection using the controller's global view.
- Device isolation and micro-segmentation; quarantine of compromised devices.
- Application vetting and isolation (sandboxing) for northbound apps.
- Logging, auditing and regular software updates.

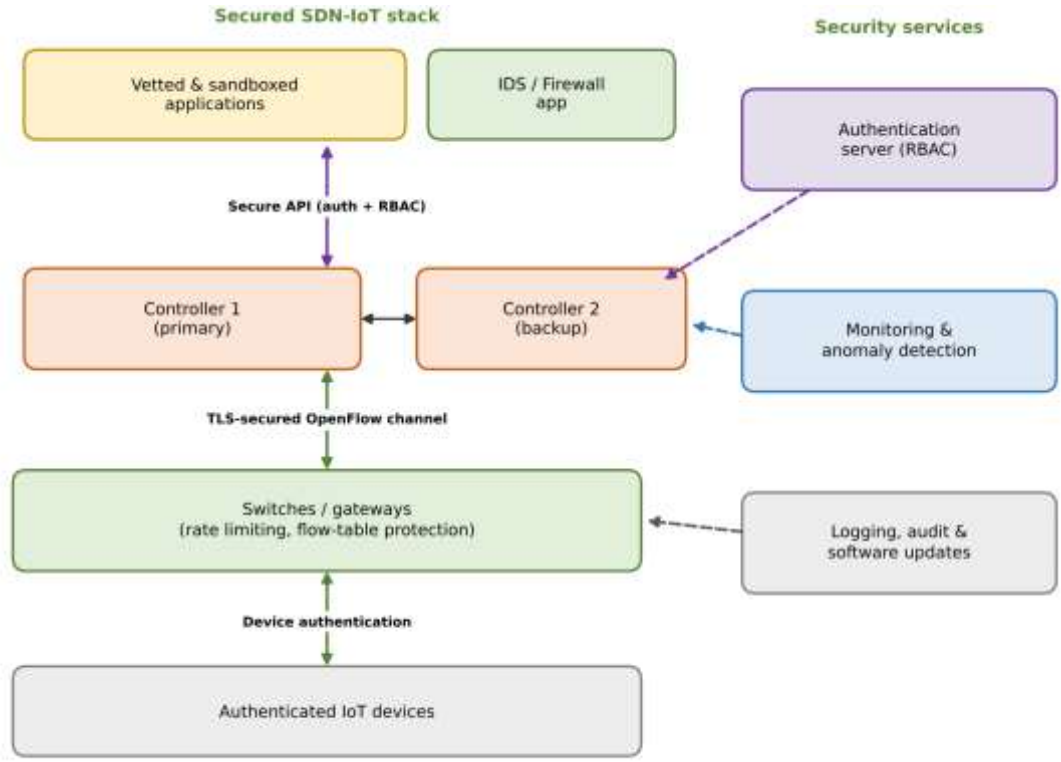


Fig 6.2: Secured SDN Architecture for IoT

6.3 Privacy

IoT data (e.g., health, location) is sensitive. The SDN controller should minimise collection of payload data, use encryption and enforce access policies so that data is accessed only by authorised parties.

7. Network Management Challenges in IoT

Network management means monitoring, configuring, controlling and maintaining the network so that it works reliably and securely (FCAPS model: Fault, Configuration, Accounting, Performance, Security). IoT makes this very difficult because of the following challenges.

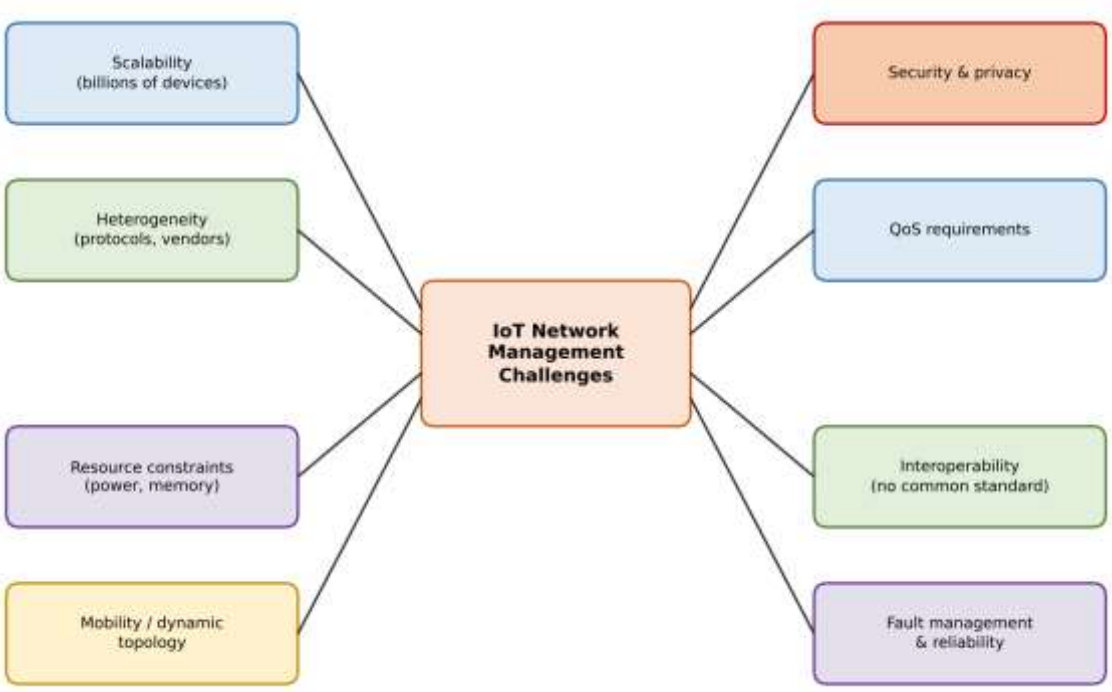


Fig 7.1: Network Management Challenges in IoT

- **Scalability:** billions of devices; traditional management (SNMP polling) cannot handle such scale.
- **Heterogeneity:** different devices, vendors, protocols (Zigbee, BLE, LoRaWAN, MQTT, CoAP) and data formats.
- **Resource constraints:** low memory, processing power and battery; heavy management protocols cannot run on devices.
- **Dynamic topology and mobility:** devices frequently join, leave, move or sleep, so topology keeps changing.
- **Security and privacy:** large attack surface, weak device security, difficult firmware updates.
- **Quality of Service (QoS):** different applications need different latency, bandwidth and reliability.
- **Interoperability:** lack of common standards and vendor lock-in.

- **Fault management:** unreliable wireless links and failures are hard to detect and localise.
- **Configuration and provisioning:** manual configuration of a huge number of devices is not practical (need zero-touch provisioning).
- **Big data and monitoring:** enormous volume of management and application data.
- **Energy efficiency:** management traffic itself drains battery.

How SDN helps with these challenges

Challenge	SDN contribution
Scalability	Centralised/distributed controllers, automated programmable configuration
Heterogeneity	Abstraction of devices; common southbound protocols; gateway-level translation
Resource constraints	Intelligence moved from devices to controller, devices need only simple forwarding
Dynamic topology	Global real-time view allows quick re-programming of paths
Security	Central policy enforcement, isolation, anomaly detection
QoS	Flow-based priority and traffic engineering

8. Integration of SDN with IoT Networks

8.1 Need for integration

Applying SDN to IoT combines IoT's massive, diverse device base with SDN's programmability and central control. However, standard OpenFlow was designed for wired Ethernet networks; IoT uses low-power, lossy wireless links and constrained devices, so some adaptation is needed.

8.2 Integrated architecture

A typical SDN-based IoT architecture has the following layers:

- **Perception / Device layer:** sensors, actuators, RFID tags, smart objects.
- **Network / Data plane:** SDN-enabled IoT gateways, access points and switches that forward data from devices.
- **Control plane:** SDN controller (possibly with edge controllers close to devices and a main controller in the cloud) managing flows, topology, resources.
- **Application / Management plane:** IoT applications (smart home, smart city, healthcare, industry) and network management apps (monitoring, security, QoS) using the northbound API.

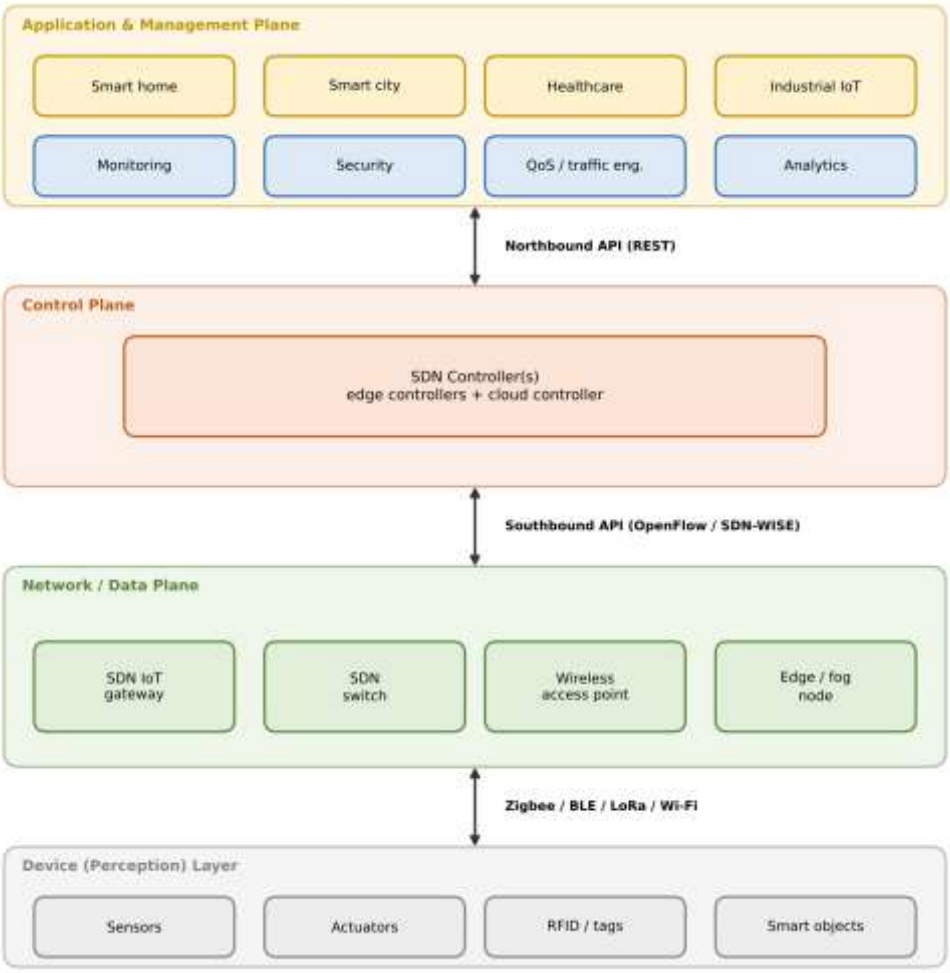


Fig 8.1: SDN-based IoT Architecture

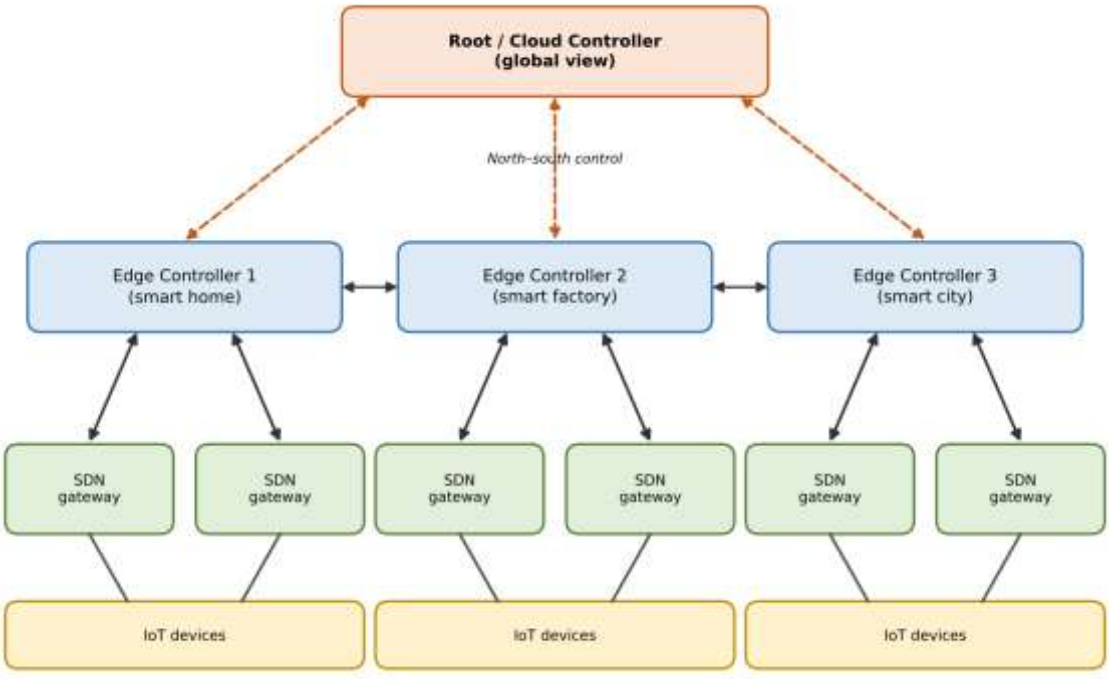


Fig 8.2: Hierarchical (Edge + Cloud) Controller Deployment for IoT

8.3 Approaches to integration

- SDN-enabled IoT gateway: the gateway acts as an OpenFlow switch; it translates between IoT protocols (Zigbee, BLE, CoAP) and IP/OpenFlow.
- Software-Defined Wireless Sensor Networks (SD-WSN): e.g., Sensor OpenFlow, TinySDN, SDN-WISE – lightweight SDN designed for low-power wireless sensor nodes.
- Edge/Fog SDN: controllers near the devices reduce latency and control traffic.
- SDN + NFV: network functions such as firewall, NAT and load balancers run as virtual functions chained by SDN.
- Network slicing: separate logical slices for different IoT services (critical, massive IoT, etc.), especially in 5G-based IoT.

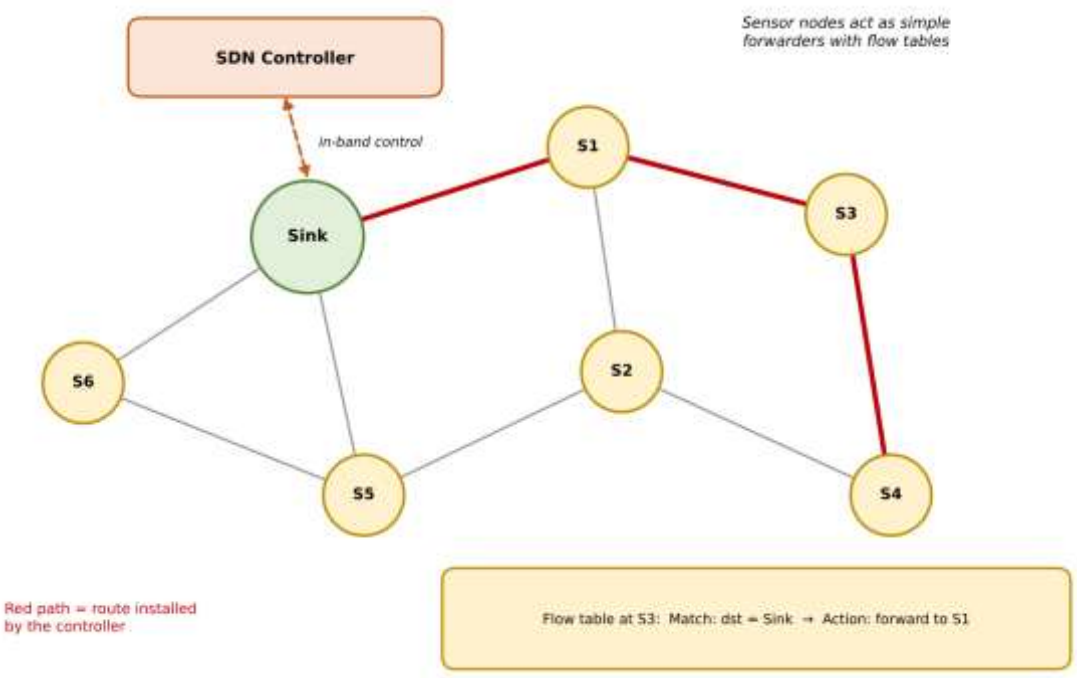


Fig 8.3: SDN-WISE / Software-Defined Wireless Sensor Network

8.4 Working (typical flow)

- IoT device sends data to the SDN-enabled gateway/switch.
- The switch checks its flow table; if a matching rule exists, the packet is forwarded accordingly.
- If there is no match, a Packet-In message is sent to the controller.
- The controller applies the policy (authentication, QoS, routing) and decides the path.
- The controller installs a flow rule (Flow-Mod) in the switches along the path.
- Following packets of that flow are forwarded directly by the data plane; statistics are sent back to the controller for management and analytics.

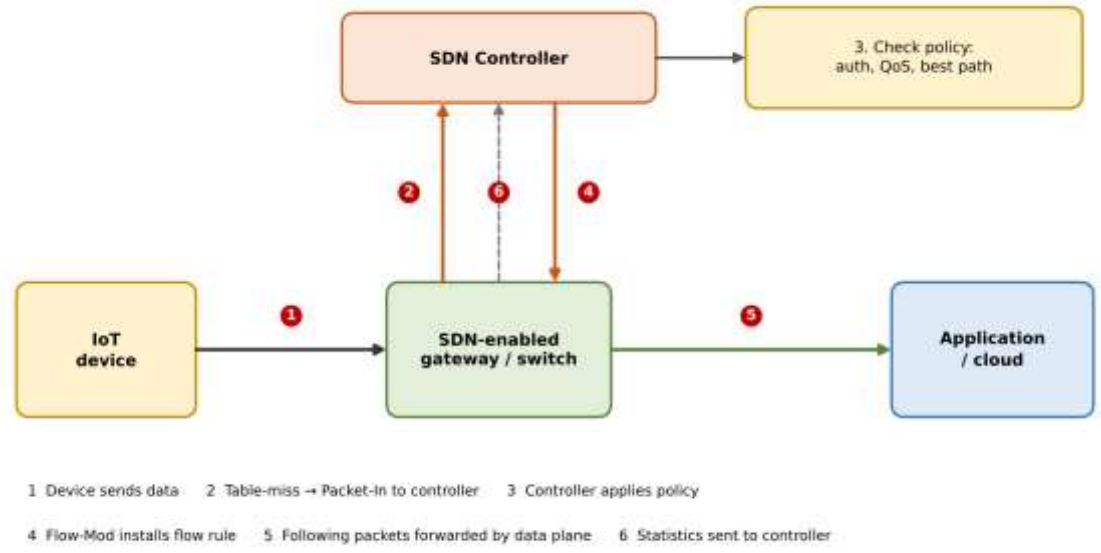


Fig 8.4: Flow Setup Procedure in SDN-based IoT

8.5 Use cases

- **Smart home:** per-device policies, isolating insecure devices, bandwidth for video cameras.
- **Smart city:** managing heterogeneous traffic, lights, parking and sensors with one controller.
- **Industrial IoT (IIoT):** low-latency, deterministic traffic and strict security.
- **Healthcare:** high-priority path for patient monitoring data, privacy enforcement.
- **Smart grid and vehicular networks:** dynamic routing and quick recovery from failures.

8.6 Challenges of integrating SDN with IoT

- Controller scalability and single point of failure.
- Control-channel overhead on low-power wireless links.
- Lack of standardised southbound protocols for constrained devices.
- Latency of controller decisions for real-time applications.
- Security of the SDN layers (Section 6).
- Interoperability with legacy IoT systems; migration complexity.
- Energy consumption in sensor nodes due to control messages.

9. Quick Revision Summary

- SDN = separation of control plane and data plane + centralised programmable controller.
- Architecture: Application plane – Northbound API – Control plane (controller) – Southbound API (OpenFlow) – Data plane (switches).
- Benefits for IoT: flexibility, centralised control, security, traffic management.

- Models: Open SDN (pure OpenFlow), API SDN (device APIs), Overlay SDN (virtual tunnels), Hybrid SDN (SDN + traditional).
- Data handling and analytics at controller: flow statistics, filtering, prioritisation, anomaly detection, ML-based prediction.
- Security: SDN gives central security but the controller and channels must be protected (TLS, authentication, redundancy).
- IoT management challenges: scalability, heterogeneity, constrained devices, mobility, security, QoS, interoperability.
- Integration: SDN-enabled gateways, SD-WSN, edge/fog controllers, NFV and network slicing.